UNIVERSIDAD NACIONAL DEL ALTIPLANO

FACULTAD DE INGENIERÍA ESTADÍSTICA E INFORMÁTICA

LENGUAJES DE PROGRAMACION

DOCENTE:

COYLA IDME LEONEL

ESTUDIANTE:

TICONA PERALTA JERALDINE ISABEL

SEMESTRE:

III

PUNO - PERÚ

2026



Lenguajes de programación

Concepto: Conjunto de reglas sintácticas, semánticas y símbolos formales que permiten a un ser humano escribir instrucciones estructuradas, para que una computadora las interprete, procese y ejecute

Elementos esenciales de un lenguaje de programación

- **Sintaxis:** Es el conjunto de reglas gramaticales que dictan como deben escribirse las palabras, símbolos y estructuras para que el código sea valido.

- **Semantica:** Es el significado o acción lógica que realiza cada instrucción escrita.

- **Vocabulario:** (Palabras reservadas) Palabras clave con significados especiales para el lenguaje que no pueden usarse para otros fines.
  Ejemplo: print, cout, while, etc

Programación

↳ Es el proceso de diseñar, escribir, probar y mantener código fuente para dar instrucciones a una computadora permitiendole ejecutar tareas específicas o resolver problemas de forma automatizada.

Algoritmo

↳ Secuencia finita, ordenada y lógica de pasos o instrucciones precisas que deben de seguirse para realizar una tarea o resolver un problema concreto.

Lenguaje de alto nivel

↳ Es un lenguaje diseñado para ser leído y escrito fácilmente por los seres humanos, utiliza palabras en ingles y estructuras lógicas cercanas al lenguaje nativo, independizandolo al programador de la arquitectura interna del hadware.
Ejemplo: C++, phyton, javascript, C++

Lenguaje de bajo nivel

↳ Es un lenguaje que opera con mínima o nula abstracción del hadware diseñado para comunicarse o interactuar directamente con la memoria y el procesador de la computadora
Ejemplo: lenguaje maquina (binario)
lenguaje Assembly (terminos mnemonicos)

Concepto

- Es una aplicación de sofware que reúne todas las herramientas necesarias para que un desarrollador escriba y pruebe código en un solo lugar. Componentes principales de un IDE:
  - Editor de código fuente: incluye resultado de sintaxis autocompletado y detección de errores en tiempo real
  - Compilador \ Interpretador: Herramienta que permite ejecutar el programa con un solo click.
  - Depurador (debugger): Herramienta para inspeccionar la memoria, avanzar línea por línea y rastrear errores.
  - Gestor de proyectos: Organiza multiples archivos, bibliotecas y configuraciones del programa

Variable:

- Una variable es un espacio reservado en la memoria ram que se utiliza para almacenar un valor que puede cambiar a lo largo de la ejecución de un programa, tiene un nombre (identificador) y un tipo de datos asociado.

Ejemplo: Convertir °C a °F

Constante:

- Una constante es un valor almacenado en memoria que no puede cambiar ni ser modificada una vez halla sido definida durante la ejecución del programa, se utilizan para cuidar, evitar errores y representar valores, fijar el dominio.

Ejemplo: P_i = 3.14151692

Tipos de datos

LENGUAJES DE PROGRAMACION 1

CATEGORIA	TIPO DE DATO	TAMAÑO HABITUAL	RANGO
ESTEROS	int	4 bytes (32 bits)	-2,147,483,648 A 2,147,483,647
	short	2 bytes (16 bits)	-32,768 a 32,767
	long	4 bytes (32 bits Windows/ 8 bytes (64 bits)) Linux	-2,147,483,648
	long long	8 bytes (64 bits)	-9,223,372,036,854,775,808
PUNTO FLOTANTE	float (Simple precisión)	4 bytes (32 bits)	Aprox. 1.2e-38 a 3.4e38
	double (Doble precisión)	8 bytes (64 bits)	Aprox. 2.3e-308 a 1.7e308

CARÁCTER	char	1 byte (8 bits)	0 a 255
	wchar	2 bytes (16 bits) en Windows / 4 bytes (32 bits) en Linux	0 a 65,583
LÓGICO	bool	1 byte	true (verdadero) o false (falso)

DFD

↳ Es una representación grafica que muestra como fluye la información a traves de un sistema o proceso.

Elementos principales :

SIMBOLO	NOMBRE	FUNCIÓN
☐ / ☐	entrada ext	Representa la fuente de origen de los datos.
○	Proceso	Operación o función que transforma los datos.
≡	Almacen de datos	Lugar donde se guardan los datos, temporal o permanente.
Flecha	Flujo de datos	Muestra la dirección en la que viaja la información entre los elementos.

Seint

↳ Es una herramienta educativa de software libre diseñada para ayudar a estudiantes a dar sus primeros pasos en la programación y el desarrollo de la lógica algorítmica.

Utiliza pseudocodigo en español, lo que permite centrarse, en los conceptos fundamentales de la programación (variables, condicionales, bucles y funciones) sin la complicación de la sintaxis escrita de un lenguaje real (como C++, Java o Python)

Actividad :

* Conversor de tiempo (segundos a minutos)
  ↳ Leer un valor en segundos e imprime a cuantos minutos y horas equivale.

Date. / /

* Calculo del IGV de un producto
  ↳ Solicita el precio base de un producto y calcula el monto del IGV (18%) junto con el precio total final.

* Área y perimetro de un rectangulo
  ↳ Pide la long de la base y la altura de un rectangulo para imprimir su area y perimetro.

Operadores

OPERADORES ARITMETICOS

+ , - , * , / , %, ++ , - -

OPERADORES RELACIONALES

== , != , > , < , >= , <=

OPERADORES LÓGICOS

& & , || , ! , AND , O , NOT

IF - else → las estructuras IF-else son sentencias de control condicionales que permiten tomar decisiones en el código ejecutado bloques de instrucción unicamente cuando se cumple una condición

Estructura IF → (condicional simple) evalua una condición. si la condición resulta verdadera (true), se ejecuta el codigo dentro del bloque. si es falsa (false), el programa emite ese bloque y continua con la siguiente instrucción

Estructura IF - else → (condicional doble) permite definir un camino alternativo, si la condición resulta verdadera (true) se ejecuta el primer bloque. si es falso (false), se ejecuta obligatoriamente el bloque dentro de else.

Switch - case → En C++, switch / case, es una estructura de control de selección multiple que evalua el valor de una variable y ejecuta un bloque de código segun la coincidencia con una lista de casos enteros o caracteres

```
switch (variable)
  {
   case var
     // Ejecuta un codigo
       break
   case var 2
     // Ejecuta un codigo
       break
   default
     // codigo : no coincide con ningun previo case
       break ;
  }
```

★ Crear un codigo C++ para la clasificación de glucosa en sangre para la detección de hipoglusemia, diabetes o prediabetes.

★ Completar hasta del 1 al 10 000 en el código de numeros a letras.

[redacted]

- También llamada bucle, ciclo o iteración. Es un bloque de codigo que permite ejecutar una o varias instrucciones de formas recurrentes mientras se cumpla una determinada condición logica.

Su propósito principal es automatizar tareas repetitivas, evitar la duplicidad de código y procesar colecciones de datos.

For (inicio ; Final ; incremento)

For (i=1 ; i<=10 ; i++)

Iteración.- Es cada una de las repeticiones o ejecuciones individuales de un bucle.
Si un bude ejecuta 5 veces, realiza 5 iteraciones.

Contador.- Es una variable entera cuyo valor se incrementa o decrementa en una cantidad constante (generalmente de 1 en 1) en cada iteración. Se utiliza para registrar la cantidad de veces que ocurre un evento o se repite un ciclo.

```
contador = contador + 1 ;

contadorPar = contadorPar + 1 ;
```

Acumulador.- Es una variable que almacena la suma o multiplicación sucesiva de valores variables en cada interación, se utiliza para obtener totales, promedios o sumatorias acumuladas.

```
Suma = suma + 1 ;

Suma += i ;
```

Control de flujo iterativo

C++ proporciona sentencias especiales para alterar la ejecución normal de una estructura repetitiva.

break : Interrumpe y sale inmediatamente del bucle, ignorando las iteraciones restantes.

continue : Salta del resto de las instrucciones de la iteración actual y pasa directamente a la siguiente.

Anidamiento basico de bucles

tabla de multiplicación

```
1 x 1 = 1      2 x 1 = 2     . . .
1 x 2 = 2      2 x 2 = 4     . . .
:              :  :
1 x 10 = 10    2 x 10 = 20   . . .  10 x 10 = 100
```

```
For (i=1 ; i<=10 ; i++)
    For (j=1 ; i<=10 ; j++)
        cout << i << "x" << j << "=" << i*j << endl
```

Funcionamiento Basico

- Ejecución obligatoria → El programa entra al ciclo y ejecuta las instrucciones del bloque, directamente, sin comprobar ninguna condición previa.

- Evaluación posterior → al llegar al final del bloque, se evalua la condición logica expresada en el while.

- Repetición → si la condición es verdadera, el flujo regresa al inicio del bloque para repetir el proceso.

- Finalización → si la condición es falsa, el ciclo termina y el programa continua con la instrucción siguiente.

Ejemplo → Leer "n" numeros enteros, obtener la suma total y el promedio de los numeros ingresados.

• Crea tabla de multiplicar con 7.

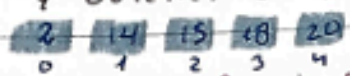
Arreglo

Un arreglo (tambien conocido como ARRAY, vector o matriz) es una estructura de datos homogenea y de tamaño fijo, que permite almacenar una coleccion de elementos del mismo tipo, bajo un unico nombre. cada elemento individual dentro del arreglo se identifica y se accede mediante uno o mas indices numericos

1. Arreglo unidimensional (vector)

- Es una colección lineal de datos que se organizan de forma secuencial, uno al lado del otro en la memoria del computador. Se puede visualizar como una sola fila con multiples casillas.

Ejemplo: Leer "n" notas, mostrar las notas ingresadas y obtener el promedio



int
int Notas [5]
notas [1] - 14

2. Arreglo / Arry bidimensional (matriz)

- Es una estructura organizada en una tabla de 2 dimensiones: filas y columnas, se puede imaginar como una cuadricula o una hoja de calculo, donde la intersección de una fila y una columna determina una posición o casilla específica.

Ejemplo matriz

1	14	15	16
	8	13	20
	0	1	2

```
int matriz [2][3];
matriz [0][1] = 15
```

Ejemplo: Leer una matriz de orden n x n y sumar sus elementos.

Calcular el factorial de un numero n

Estructuras repetitivas while

- (mientras) es un bucle o ciclo de control que ejecuta un bloque de código de forma iterativa mientras se cumpla una condición lógica determinada.

A diferencia de otros bucles (como for) el bucle while se usa principalmente cuando no se conoce con exactitud el número exacto de iteraciones que se van a realizar antes de iniciar la ejecución.

FUNCIONAMIENTO BÁSICO

1. Evaluación.- Antes de ejecutar el código, se evalúa la condición lógica.

2. Ejecución.- Si la condición es verdadera (true) se ejecuta el cuerpo del bucle

3. Repetición.- Al terminar el bloque, se vuelve a evaluar la condición y el proceso continúa hasta que la condición pase a ser falsa (false)

4. Finalización.- Si la condición es falsa desde el inicio el bloque dentro del bucle no se ejecutará ni una sola vez.

do-while

- (hace-mientras) es un bucle o ciclo de control que ejecuta un bloque de código al menos una vez y luego continúa repitiendolo mientras se cumpla una condición lógica determinada

A diferencia del bucle while tradicional, la condición de evaluación se encuentra al final del bloque de código.